



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Modelos de procesos de Desarrollo de Software

TEMA:

Evaluación de modelo de procesos de desarrollo de software

NRC

30708

INTEGRANTES

- Kevin Anthony Chalan Yamberla
- Valeria Naomi Guaiguacundo Aguirre
- Kevin Jahir Sivinta Alomoto
- Nicolás Alejandro Zaldumbide Armas

FECHA:

12/05/2026

Docente:

ING. COLA PEREZ CRISTIAN ANDRES



Índice

1. Información general del proyecto.....	3
2. Modelo de proceso y metodología de trabajo.....	3
3. Planificación del proyecto.....	4
4. Gestión de requerimientos.....	7
5. Casos de uso.....	12
6. Evidencias del proceso de desarrollo.....	14
7. Entregable Funcional.....	15
8. Relación con CMMI.....	15
9. Conclusiones.....	16

1. Información general del proyecto

1. *Nombre del proyecto*

“Viveres Danielito”

2. *Integrantes del grupo*

- Kevin Anthony Chalan Yamberla
- Valeria Naomi Guaiguacundo Aguirre
- Kevin Jahir Sivinta Alomoto
- Nicolás Alejandro Zaldumbide Armas

3. *Objetivo del sistema*

Mejorar la organización y el control del stock mediante el registro digital de productos, entradas y salidas de inventario, facilitando el uso por parte del propietario del negocio a través de una interfaz sencilla.

4. *Descripción breve del problema a resolver*

La tienda “Viveres Danielito”, ubicada en Amaguaña, actualmente no cuenta con un sistema automatizado para el control de su inventario. El registro de productos y las existencias se realiza de manera manual mediante una libreta, donde se anotan los productos ingresados y la información de los proveedores.

Este método provoca desorganización y aumenta la posibilidad de tener errores como pérdidas de información, registros incompletos o dificultades para entender algunos números.

Debido a estos problemas se requiere desarrollar un sistema que facilite el acceso a la información y mejore la organización en la tienda.

2. Modelo de proceso y metodología de trabajo

1. *Modelo de proceso utilizado*

El proyecto se basa en un modelo **iterativo e incremental**, desarrollando el sistema por ciclos divididos en sprints, en los que se implementan los requisitos previamente obtenidos de la reunión con el cliente. Cada iteración da como resultado un incremento funcional para el sistema, lo que permitió al equipo trabajar en el proyecto de forma modular, facilitando la retroalimentación constante y la detección temprana de errores. Este enfoque coincide con lo expuesto por Pressman (2010, pp. 45-46), quien afirma que este modelo permite que el software se adapte mejor a las necesidades del usuario conforme se aprende entre un lanzamiento y otro.

2. *Metodología aplicada*

El proyecto aplicó la **metodología ágil Scrum**, lo que permitió adaptarse a los cambios emergentes en el proyecto (ya sea redefinición de requisitos o nuevas funcionalidades

La aplicación de esta metodología permite que el equipo pueda mantener una sincronización a través de reuniones de planificación y el uso de GitHub para cumplir con el seguimiento de cambios.

Se eligió el modelo incremental debido a que permite desarrollar el sistema por incrementos o sprints en este caso, facilitando la mejora continua y el control del avance del proyecto de una forma precisa.

Cada incremento añade nuevas funcionalidades al sistema, permitiendo detectar errores en fases tempranas y realizar las correcciones necesarias. El modelo facilita la retroalimentación del cliente y asegura que el sistema se adapte a las necesidades reales del sistema.

- El proyecto se organizó mediante un enfoque incremental, utilizando la metodología ágil Scrum para repartir las funcionalidades en sprints y con ellas presentar entregables.
- El sistema está basado en el patrón de arquitectura MVC (Modelo-Vista-Controlador), separando la interfaz, la lógica y el manejo de datos, lo que permite mantener una mejor organización y estructura en el proyecto.
- Para organizar las tareas y asignarlas a los miembros del equipo se utilizó Jira, donde se controló el estado de los requerimientos.
- También se utilizó GitHub para guardar el código del sistema y que todos los miembros del equipo puedan visualizarlo. La documentación y anexos del proyecto también fueron almacenados en este repositorio.
- Para la asignación de roles, reuniones con el cliente y socialización de entregables se realizaron reuniones mediante Google Meet, cuyas evidencias y actas se encuentran anexadas en el repositorio.

1. Cronograma de trabajo

[illegible]

[illegible][illegible]

2. *Sprint o planificación parcial*

EDSTVD Sprint 1 20 Apr – 10 May (6 work items)		Complete sprint	...
EDSTVD-85	REQ001 Crear las clases para modelo, vista y controlador para Inicio de Sesión con las validaciones pertinentes.	FINALIZED	- NA
EDSTVD-86	REQ001 Conectar el programa a MongoDB.	FINALIZED	- NA
EDSTVD-87	REQ001 Crear la GUI para inicio de sesión y vincularlo al controlador.	FINALIZED	- NA
EDSTVD-88	REQ002 Crear las clases para modelo, vista y controlador para Registro de Producto con las validaciones pertinente.	FINALIZED	- PS
EDSTVD-89	REQ002 Vincular las clases nuevas a la conexión de base de datos.	FINALIZED	- PS
EDSTVD-90	REQ002 Crear la GUI para Registrar Producto y vincularlo al controlador.	FINALIZED	- PS



EDSTVD Sprint 2 11 May – 31 May (11 work items)			Start sprint	...
EDSTVD-97	REQ003 Crear la interfaz para buscar producto	TO DO	-	15
EDSTVD-98	REQ003 Añadir los filtros de búsqueda: nombre, existencias, proveedor.	TO DO	-	15
EDSTVD-99	REQ003 Conectar la aplicación a MongoDB para la búsqueda	TO DO	-	15
EDSTVD-100	REQ004 Crear interfaz para ingreso de stock	TO DO	-	NA
EDSTVD-101	REQ004 Crear el formulario para ingreso de stock y conectarlo a la base de datos	TO DO	-	NA
EDSTVD-102	REQ005 Crear interfaz para salida de stock	TO DO	-	NA
EDSTVD-103	REQ005 Crear el formulario para salida de stock y conectarlo a la base de datos	TO DO	-	NA
EDSTVD-104	REQ006 Crear la interfaz para visualizar los movimientos realizados	TO DO	-	15
EDSTVD-105	REQ006 Listar los movimientos realizados conectando con la base de datos, filtrados por fecha	TO DO	-	15
EDSTVD-106	REQ006 Añadir la opción de anular el movimiento dando click derecho sobre el registro	TO DO	-	15
EDSTVD-107	REQ006 Añadir una doble confirmación para anular, verificar que se reflejen los cambios en la base de datos	TO DO	-	15

EDSTVD Sprint 3 1 Jun – 21 Jun (6 work items)			Start sprint	...
EDSTVD-108	REQ007 Generar interfaz para eliminar/modificar un producto	TO DO	-	15
EDSTVD-109	REQ007 Conectar con la base de datos y generar un listado de los productos existentes	TO DO	-	15
EDSTVD-110	REQ007 Añadir la opción de eliminar el producto dando click derecho sobre él	TO DO	-	15
EDSTVD-111	REQ007 Añadir doble confirmación para eliminar un producto, verificar que los cambios se reflejen en la base de datos	TO DO	-	15
EDSTVD-112	REQ008 Permitir modificar los atributos nombre, precio y proveedor de la lista de productos ya programada	TO DO	-	NA
EDSTVD-113	REQ009 Añadir opción de reporte a txt utilizando la lista de productos previamente programada	TO DO	-	NA

3. Distribución de tareas y responsables

Nombre	Valeria Guaiguacundo
Rol	Product Owner
Categoría Profesional	Programador de Desarrollo
Responsabilidad	Verificar y controlar el encaminamiento del trabajo, priorizar el backlog del producto y validar las entregas con el cliente final: Enrique Guaiguacundo
Información de contacto	vnguaiguacundo@espe.edu.ec

Nombre	Kevin Chalan
Rol	Scrum Master
Categoría Profesional	Programador de Desarrollo
Responsabilidad	Analizar los requerimientos del software, liderar al equipo de trabajo en la metodología Scrum y asegurar el cumplimiento de los hitos del proyecto.
Información de contacto	kachalan@espe.edu.ec

Nombre	Jahir Sivinta
Rol	Team Member (Developer)
Categoría Profesional	Programador de Desarrollo
Responsabilidad	Diseñar y programar la lógica de negocio para la búsqueda, modificación de productos y generación de reportes TXT.
Información de contacto	kjsivinta@espe.edu.ec

Nombre	Nicolás Zaldumbide
Rol	Team Member (Developer)
Categoría Profesional	Programador de Desarrollo
Responsabilidad	Diseñar y programar las funcionalidades de login y las que permitan agregar y eliminar productos, así como las de entrada (stock-in) y salida (stock-out) de productos del inventario.
Información de contacto	nazaldumbide@espe.edu.ec

4. Gestión de requerimientos

1. *Requerimientos funcionales*

Identificación	RF01 (REQ 001)
Nombre	Iniciar Sesión
Actor	Manager
Precondición	El sistema muestra la interfaz de ingreso de credenciales.
Descripción	Permite al usuario acceder a las funcionalidades del sistema mediante las credenciales
Prioridad	Alta

Identificación	RF02 (REQ 002)
Nombre	Registrar producto
Actor	Manager

Precondición	El usuario debe haber iniciado sesión con éxito.
Descripción	Permite ingresar un nuevo producto al sistema, con atributos como nombre, proveedor, tipo y precio unitario.
Prioridad	Alta

Identificación	RF03 (REQ 003)
Nombre	Buscar producto
Actor	Manager
Precondición	El usuario debe haber iniciado sesión con éxito.
Descripción	Permite visualizar la información de un producto (nombre, existencias, proveedor) en una tabla, ingresando un término de búsqueda.
Prioridad	Media

Identificación	RF04 (REQ 004)
Nombre	Registrar entrada de inventario
Actor	Manager
Precondición	El usuario debe haber iniciado sesión con éxito.
Descripción	Permite registrar el ingreso de mercadería aumentando el stock existente mediante el nombre del producto y rellenar los campos con la información.
Prioridad	Alta

Identificación	RF05 (REQ 005)
Nombre	Registrar salida de inventario
Actor	Manager
Precondición	El usuario debe haber iniciado sesión con éxito.

Descripción	Registra el retiro por venta, pérdida o vencimiento de productos, disminuyendo la cantidad en el inventario.
Prioridad	Alta

Identificación	RF06 (REQ 006)
Nombre	Anular movimiento
Actor	Manager
Precondición	El usuario debe haber iniciado sesión con éxito.
Descripción	Borra un movimiento de producto de forma definitiva tras una doble confirmación, ya sea un registro de entrada o salida.
Prioridad	Alta

Identificación	RF07 (REQ 007)
Nombre	Eliminar producto
Actor	Manager
Precondición	El usuario debe haber iniciado sesión con éxito.
Descripción	Borra definitivamente un producto y sus registros tras una doble confirmación.
Prioridad	Baja

Identificación	RF08 (REQ 008)
Nombre	Modificar producto
Actor	Manager
Precondición	El usuario debe haber iniciado sesión con éxito.
Descripción	Permite modificar los atributos del producto (nombre, precio, proveedor).
Prioridad	Baja

Identificación	RF09 (REQ 009)
----------------	----------------

Nombre	Generar reporte de disponibilidad
Actor	Manager
Precondición	El usuario debe haber iniciado sesión con éxito.
Descripción	Genera automáticamente un archivo llamado productos.txt con la lista actualizada del inventario.
Prioridad	Media

2. *Requerimientos no funcionales*

ID	Nombre	Descripción
RNF01	Interfaz del sistema	Debe ser intuitiva, sencilla y basada en una interfaz gráfica de usuario (GUI)
RNF02	Rapidez	El sistema debe procesar las actualizaciones de stock de manera fluida y sin demoras.
RNF03	Integridad	No se permiten cantidades negativas ni caracteres numéricos en el nombre del producto.
RNF04	Portabilidad	El esquema debe funcionar correctamente en diferentes sistemas operativos manteniendo la misma estructura

3. *Evidencias de levantamiento de información*

Esta información se encuentra en la Acta 01 en el anexo [1]

4. *Relación entre requerimientos y funcionalidades del sistema*

REQUISITO	Descripción	Funcionalidad Sistema
-----------	-------------	-----------------------

REQ001 - Iniciar Sesión

Permite al usuario acceder a las funcionalidades del sistema mediante las credenciales

REQ002 - Registrar Producto

Permite ingresar un nuevo producto al sistema, con atributos como nombre, proveedor, tipo y precio unitario.

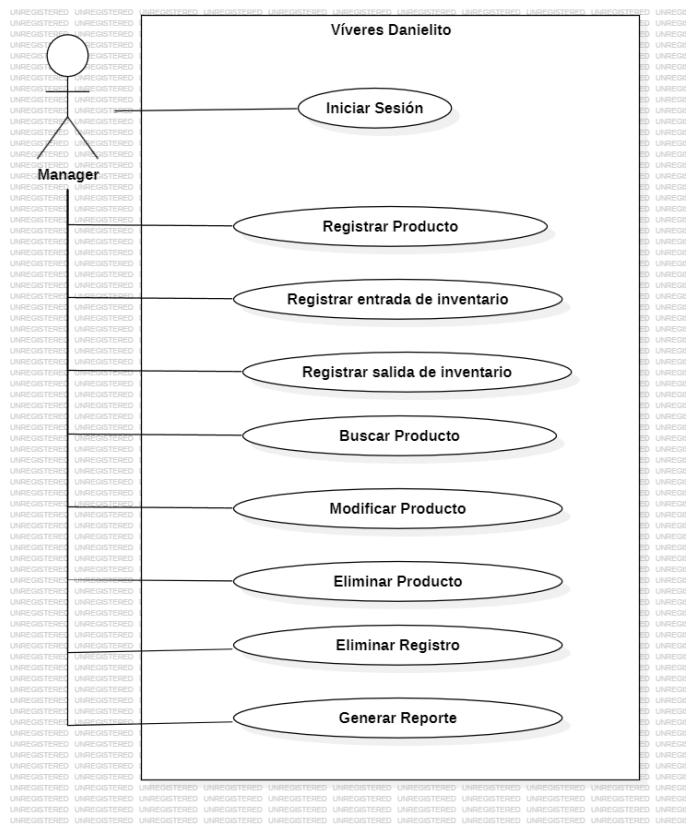
	Pr... x + Valeria > ViveresDanielito > Producto Documents 1 Aggregations Schema Indexes 1 Validation Type a query: { field: 'value' } or Generate query ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODI <pre>{ "_id": ObjectId("6a82a414b5d3153a4f57fb85"), "nombre": "leche entera", "tipo": "lacteos", "precio unitario": 8.8, "proveedor": "Vita" }</pre>
--	--

5. Casos de uso

1. Identificar actores

Actor	Descripción
Manager	Es el usuario final del sistema encargado de gestionar el inventario, registrar si un producto tiene entradas o salidas y emitir reportes

2. Elaborar diagramas de caso de uso



Nota.- Diagrama de Casos de Uso

3. Describir los casos

<i>Caso de Uso</i>	<i>Actor</i>	<i>Condición</i>	<i>Descripción</i>	<i>Prioridad</i>
Iniciar Sesión	Manager	Conocer las credenciales de acceso por defecto	El sistema valida el usuario y contraseña, en esta versión del sistema las credenciales están quemadas para inicializar el sistema	Alta
Registrar Producto	Manager	Haber iniciado sesión con éxito	permite añadir un nuevo producto al inventario ingresando: nombre, proveedor, tipo y precio en la base de datos	Alta
Registrar entrada de inventario	Manager	Haber iniciado sesión y que el producto ya esté registrado.	Permite el ingreso de un producto para aumentar la cantidad de stock disponible en el inventario.	Alta
Registrar salida de inventario	Manager	Haber iniciado sesión y que el producto tenga stock disponible.	Permite registrar cuando un producto sale para restar esa cantidad del inventario.	Alta
Anular movimiento	Manager	Debe existir un movimiento de entrada o salida registrado previamente.	Permite anular un movimiento que se haya ingresado por error, pidiendo una confirmación antes de borrar el registro.	Alta
Buscar Producto	Manager	Haber iniciado sesión con éxito	Permite buscar y visualizar la información de un producto específico mediante su nombre mostrando los resultados en una tabla	Alta
Modificar Producto	Manager	El producto debe existir en el sistema	Permite modificar los atributos que tenga nuestro producto existente.	Media
Eliminar Producto	Manager	El producto debe existir en el sistema	Permite borrar un producto del inventario trans un mensaje de confirmación de dicha acción	Media
Generar Reporte	Manager	Haber iniciado sesión con éxito	Generar un archivo txt con la lista actualizada del inventario de la tienda	Baja

6. Evidencias del proceso de desarrollo

1. Reuniones de trabajo

Establecimos algunas reuniones vía Google Meet y establecimos constancia en nuestras actas el cual puede encontrarlo en nuestro drive adjunto en los Anexos [1].

- En el Acta 01 existe constancia de la reunión con nuestro cliente por medio de una entrevista con preguntas específicas sobre sus necesidades
- En el Acta 02 consta el asignamiento de roles según la metodología Scrum de acuerdo a nuestras habilidades y destrezas para un mejor desempeño.
- En el Acta 03 cerramos nuestro primer Sprint porque de acuerdo a nuestra planificación ya concluimos con dichas tareas cada uno de nosotros hasta este Sprint.

2. Avances del sistema

Para llevar el control de nuestro progreso utilizamos JIRA, el cual pueden revisar en el enlace del Anexo [3]. Ahí se evidencia que logramos completar nuestro Sprint 1 en las fechas del 20 de abril al 10 de mayo terminando todas las tareas.

En este sprint se trabajó en la interfaz de inicio de sesión, el registro de productos y la conexión a la base de datos.

3. Versiones funcionales

El código fue desarrollado en Java usando la arquitectura MVC Modelo-Vista-Controlador. Todas las versiones funcionales y el código fuente se subieron al repositorio de GitHub que está en el Anexo [2]. Ahora mismo, las funciones disponibles del programa ya se comunican correctamente con los controladores y mandan la información a la base de datos sin problema.

4. Organización de tareas

Para priorizar las tareas se creó un Product Backlog en Excel, que también pueden encontrar guardado en la carpeta de Drive del Anexo [1]. En ese documento se listaron todos los requerimientos del REQ001 al REQ009 y el tiempo estimado que iba a tomar cada funcionalidad.

5. Control de cambios

Durante este Sprint 1 el cambio más importante que documentamos fue la actualización de nuestra documentación de requisitos para incluir la seguridad del sistema:

Cambio	Motivo	Aprobación
Se actualizó el documento ERS a su tercera versión para agregar formalmente el inicio de sesión REQ001 se puede revisar en el Anexo [2]	Se sugirió al cliente que por temas de privacidad y seguridad con su negocio era mejor implementar un sistema de Login por temas de precaución	Aprobado por el equipo y el Product Owner.

7. Entregable Funcional

A este punto, el programa compila y cumple con los objetivos trazados para el Sprint 1. El sistema ya cuenta con la arquitectura MVC implementada y permite ingresar mediante un inicio de sesión seguro. Además, ya permite el registro exitoso de nuevos productos desde su interfaz gráfica, guardando toda esta información de manera real en nuestra base de datos local de MongoDB (el código fuente de este entregable está en el Anexo [2]).

8. Relación con CMMI

1. *¿Qué prácticas de organización están aplicando?*

El equipo implementa prácticas de planificación y seguimiento del proyecto, asignando tareas mediante herramientas colaborativas y estableciendo reuniones periódicas para revisar avances. Se aplican controles de versión y gestión de cambios para mantener el crecimiento y desarrollo.

2. *¿Cómo gestionan el trabajo?*

Se utiliza una estructura basada en roles definidos y en un flujo de trabajo iterativo que permite evaluar entregables funcionales en cada ciclo, las tareas se organizan según prioridades y dependencias, garantizando una comunicación constante entre los miembros del equipo.

3. *¿Qué evidencias manejan?*

Las principales evidencias son las actas de reuniones, reportes de progreso, versiones funcionales del sistema, documentación técnica, y registros de control de cambios. Estos elementos demuestran la aplicación de buenas prácticas de ingeniería de software y la mejora continua del proceso.

¿En qué nivel de madurez consideran que se encuentra el proyecto?

El proyecto se encuentra en el nivel de madurez 2 (Gestionado) del modelo CMMI, ya que como se establece en este modelo: “un proceso de nivel de capacidad 2 puede planificarse y monitorizarse de la misma forma que cualquier proyecto” (Equipo del Producto CMMI, 2010, pp. 35–37).

Este nivel se evidencia en estos artefactos: ERS y Product Backlog (para gestión de requerimientos), actas de reunión, Jira y Sprint 1 (para seguimiento).

Se identifican prácticas con aspiración a ser nivel 3 (como la documentación detallada de requisitos), pero como el modelo indica este nivel demanda que los procesos se adapten a partir de una estandarización en los procesos de la organización, lo que no aplica en el alcance del proyecto.

Si bien el CMMI en su interpretación organizacional requiere un conjunto de procesos estándar a nivel institucional, en proyectos académicos y de desarrollo con equipo propio, la *organización efectiva es el equipo mismo*. En este caso, el proyecto cuenta con una ERS formal, un Product Backlog estructurado y actas de reunión sistemáticas, lo que constituye un conjunto de procesos definidos y replicables que guían cada iteración. Esto se alinea con el criterio de que "los procesos de nivel 3 se describen más rigurosamente que los de nivel 2" (CMMI, 2010, p. 39).

9. Conclusiones

1. *Dificultades encontradas*

Entre las principales dificultades se identificaron la coordinación de tareas entre distintos miembros del equipo, la gestión de tiempos en los entregables y la adaptación a los cambios en los requerimientos del sistema.

2. *Cómo ha influido la organización del proyecto*

La organización estructurada del trabajo permitió mejorar la comunicación, reducir errores en las versiones funcionales y mantener un control más preciso sobre los avances. Gracias a la aplicación de prácticas alineadas con CMMI, el equipo logró mayor eficiencia y calidad en los resultados.

3. *Qué aspectos deben mejorar en el proyecto*

Es necesario fortalecer la documentación técnica y la planificación de pruebas para asegurar una validación más completa del sistema. También se recomienda optimizar la gestión de tareas mediante herramientas colaborativas y definir métricas de seguimiento que permitan evaluar el progreso de manera más objetiva en futuras iteraciones.

10. Anexos

[1] Carpeta de Drive (Documentación del proyecto):

https://drive.google.com/drive/folders/13Uo0yfNSaelfW6PPSkUSYYvgVC7IVNI9?usp=drive_link

[2] Repositorio de GitHub

https://github.com/Nasakolai/30708_CGSZ_Modelos.git

[3] Tablero de Jira (Gestión de Sprints y Tareas):

<https://espe-team-modelosdeprocesos.atlassian.net/jira/software/projects/EDSTVD/boards/2?atlOrigin=eyJpIjoiYTVmYjVhNGE2NzVjNDFlZGFmZjI4Yzg5OTI4NGUxZjliLCJwIjoiajI9>

11. Bibliografía

- Equipo del Producto CMMI. (2010). *CMMI para desarrollo, versión 1.3* (Reporte Técnico CMU/SEI-2010-TR-033, pp. 35–44). Software Engineering Institute, Carnegie Mellon University.
- Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7ª ed.). McGraw-Hill Interamericana.

RÚBRICA ANALÍTICA – 7 PUNTOS

Criterio	Excelente	Medio	Bajo
Organización y planificación del proyecto (1.5 puntos)	El proyecto presenta cronograma, roles y planificación claramente estructurados y coherentes	Presenta planificación parcial o limitada	No existe organización clara del trabajo
Gestión de requerimientos y casos de uso (2 puntos)	Requerimientos claros, coherentes y correctamente relacionados con diagramas de casos de uso	Existen inconsistencias parciales entre requerimientos y diagramas	No se evidencia análisis adecuado del sistema
Evidencias y documentación del proceso (1.5 puntos)	Presenta evidencias claras, organizadas y relacionadas con el avance real del proyecto	Evidencias incompletas o poco organizadas	No se evidencian avances o documentación suficiente
Entregable funcional del sistema (1 punto)	El sistema presenta funcionalidades operativas acordes a la documentación	El sistema presenta avances limitados o inconsistentes	No existe entregable funcional verificable
Aplicación del enfoque CMMI (1 punto)	Relaciona correctamente planificación, documentación y evidencias con niveles 2–3 de CMMI	Relación superficial o parcialmente correcta	No comprende la relación con CMMI